UNIVERSITY OF PISA

Lesson 4

Silvia Giulia Galfrè

Local and global aligment

DYNAMIC PROGRAMMING

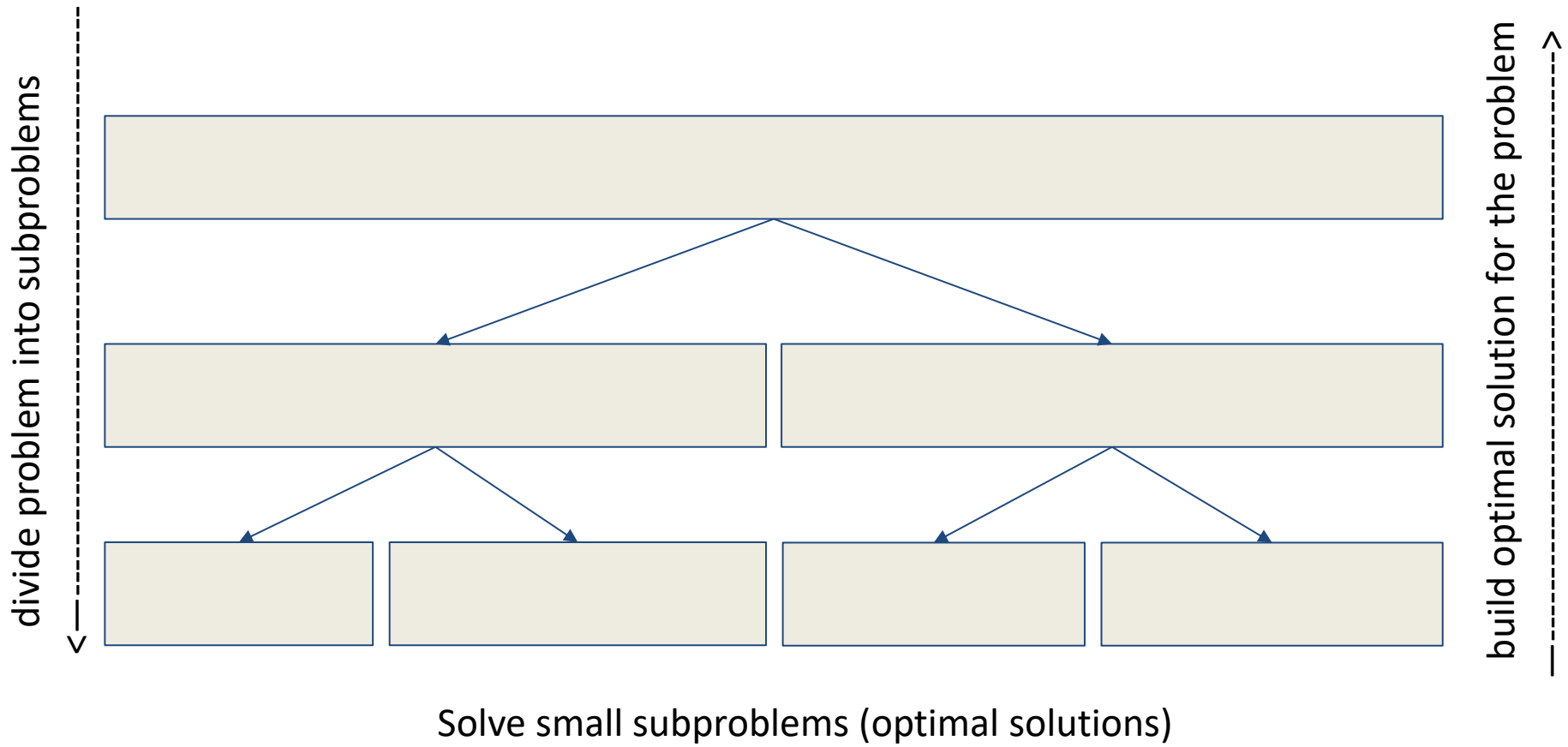
Dynamic programming

- It is a general class of algorithms based on a divide-and-conquer approach
- The solution for a small sub-problem is found and used recursively to solve the whole problem

Dynamic programming

- Optimisation algorithm
  - Objective: find the solution that maximises the score (optimal solution)
  - Main idea:
    - divide the **problem** into smaller **subproblems**
      - only for problems with *optimal substructures*
      - we can obtain the optimal solution for the **problem** by combining the optimal solutions for **subproblems**
    - find the optimal solutions for the subproblems
    - build the solution for the problem from those for the subproblems
- Very widely used in computer science

Dynamic programming



Global alignment with The Needleman-Wunsch Algorithm

- Aim: align two sequence A and B
- Len(A) = n, len(B) = m
- $A = a_1 a_2 \ldots a_n$
- $\mathrm{B} = \mathrm{b}_1 \mathrm{b}_2 \ldots \mathrm{b}_\mathrm{m}$
- We create a S matrix with A on rows and B on column to store the optimal alignment for the sub-sequence

		b_1	b_2	b_3	b_4	b_5	b_6
		Gap	H	G	W	A	G
a_1	gap						
a_2	P						
a_3	H						
a_4	S						
a_5	W						
a_6	G						

Global alignment with The Needleman-Wunsch Algorithm

- $S_{i,j}$ = optimal score for A[0:i-1] and B[0:j-1]
- $S_{0,0} = 0$
- We consider only constant gap penalty
- First row and column contains only gaps
- If gap penalty = -8

			b_1	b_2	b_3	b_4	b_5
		Gap	H	G	W	A	G
	gap	0					
a_1	P						
a_2	H						
a_3	S						
a_4	W						
a_5	G						

Global alignment with The Needleman-Wunsch Algorithm

$$S_{i,j} = max\{$$
$$S_{i-1,j-1} + sm(a_i, b_j),$$
$$S_{i-1,j} + g\ ,$$
$$S_{i,j-1} + g\}$$

$$\forall 0 < i \leq n, 0 < j \leq m$$

Where sm is the substitution matrix (in this case BL62) and g the gap penalty

			b_1	b_2	b_3	b_4	b_5
		Gap	H	G	W	A	G
	gap	0	-8	-16	-24	-32	-40
a_1	P	-8					
a_2	H	-16					
a_3	S	-24					
a_4	W	-32					
a_5	G	-40					

Global alignment with The Needleman-Wunsch Algorithm

$$S_{1,1} = max\{ S_{0,0}(0) + sm(\mathrm{H},\mathrm{P})(-2), S_{0,1}(-8) + (-8), S_{1,0}(-8) + (-8)\}$$

$$S_{2,1} = max\{ S_{1,0}(-8) + sm(\mathrm{H},\mathrm{H})(8), S_{1,1}(-2) + (-8), S_{2,0}(-16) + (-8)\}$$

			b_1	b_2	b_3	b_4	b_5
		Gap	H	G	W	A	G
	gap	0	-8	-16	-24	-32	-40
a_1	P	-8	-2				
a_2	H	-16	0				
a_3	S	-24					
a_4	W	-32					
a_5	G	-40					

Global alignment with The Needleman-Wunsch Algorithm

- This give us the score (9) but to get the alignment we need the "route"
- *Trace-back-matrix* with three symbols

			b_1	b_2	b_3	b_4	b_5
		Gap	H	G	W	A	G
	gap	0	-8	-16	-24	-32	-40
a_1	P	-8	-2	-10	-18	-25	-33
a_2	H	-16	0	-4	-12	-20	-27
a_3	S	-24	-8	0	-7	-11	-19
a_4	W	-32	-16	-8	11	3	-5
a_5	G	-40	-24	-10	3	11	9

–HGWAG
PHSW–G

Local Alignment with the Smith-Waterman Algorithm

- No change in the score but only a partial alignment is acceptable
- When a negative score is obtained, we can restart the alignment with a 0 as alternative score

		b_1	b_2	b_3	b_4	b_5	b_6
		Gap	H	G	W	A	G
a_1	gap						
a_2	P						
a_3	H						
a_4	S						
a_5	W						
a_6	G						

Local Alignment with the Smith-Waterman Algorithm

$$S_{i,j} = max\{$$
$$S_{i-1,j-1} + sm(a_i, b_j),$$
$$S_{i-1,j} + g\ ,$$
$$S_{i,j-1} + g,$$
$$0\}$$

$$\forall 0 < i \leq n, 0 < j \leq m$$

		b_1	b_2	b_3	b_4	b_5	b_6
		Gap	H	G	W	A	G
a_1	gap	0	0	0	0	0	0
a_2	P	0					
a_3	H	0					
a_4	S	0					
a_5	W	0					
a_6	G	0					

Local Alignment with the Smith-Waterman Algorithm

The *trace-back-matrix* now can have four possibility: the three as before and the fourth which means *alignment terminated.*

		b_1	b_2	b_3	b_4	b_5	b_6
		Gap	H	G	W	A	G
a_1	gap	0	0	0	0	0	0
a_2	P	0	0	0	0	0	0
a_3	H	0	8	0	0	0	0
a_4	S	0	0	8	0	1	0
a_5	W	0	0	0	19	11	3
a_6	G	0	0	6	11	19	17

Local Alignment with the Smith-Waterman Algorithm

Note: the best alignment score can be in any point of the matrix and not just in the bottom right corner

HGWA
HSWG

HGW
HSW

		b_1	b_2	b_3	b_4	b_5	b_6
		Gap	H	G	W	A	G
a_1	gap	0	0	0	0	0	0
a_2	P	0	0	0	0	0	0
a_3	H	0	8	0	0	0	0
a_4	S	0	0	8	0	1	0
a_5	W	0	0	0	19	11	3
a_6	G	0	0	6	11	19	17

Homework

- Implement the Needleman-Wunsch Algorithm
- Implement the Smith-Waterman Algorithm
- Check with Biopython that your functions return the right values and alligments

Searching sequences in databases

BLAST

Heuristic alignment algorithms

- Dynamic programming
  - Guaranteed to find the optimum alignment
  - Complexity O(nm) - Prohibitive for long sequences and multiple alignments
- Heuristic algorithms
  - Lower complexity but will find approximate matches only (good enough alignments)
  - It can be up to 100 times faster
  - Drawback: do not guarantee to find the best possible solution

BLAST - Basic Local Alignment Search Tool

- Idea:
  - Any alignment will have a short series of perfectly matching substrings
  - Start from those “seeds” and expand the local alignment to obtain longer alignments
- Steps:
  1. Remove regions of low complexity (e.g. sequence repeats)
  2. Obtain all “words” of length *w* occurring in the query sequence
  3. For each “word”, compile a list of all possible words of size *w* that can be defined in the allowed alphabet whose local alignment score is higher that a threshold *T*
  4. Search the database with all the words from the last step: they will be the matches
  5. Extend all hits in both directions
  6. Select from the previous step the highest score
- *T* and *w* are quite important parameters

BLAST after 1997

- The extensions is more demanding
- It requires two near hits with a score > *T* separated by a distance smaller than a parameter.
- Less extensions and more efficiency
- It allows gaps

Blast on NCBI

- https://blast.ncbi.nlm.nih.gov/Blast.cgi
- Search databases for sequences that align to a query sequence
- No programming experience required
- Steps:
  - Upload the query sequence
  - Set alignment parameters
    - Scoring, seed size, number of results and significance thresholds
  - (Wait and) Visualise results

Principal functions

- BLASTN
  - For nucleotides
  - It can optimize for highly similar sequences
  - Both words size and gaps penalties can be modified
    - Default
      - w = 11
      - Match/mismatch = 2 / -3
      - Gap opening and extension = -5 and -2

Principal functions

- BLASTP
  - For proteins
  - Both words size and gaps penalties can be modified
    - Default
      - w = 6
      - Substitution matrix = BLOSUM62
      - Gap opening and extension = -11 and -1

Other components of BLAST

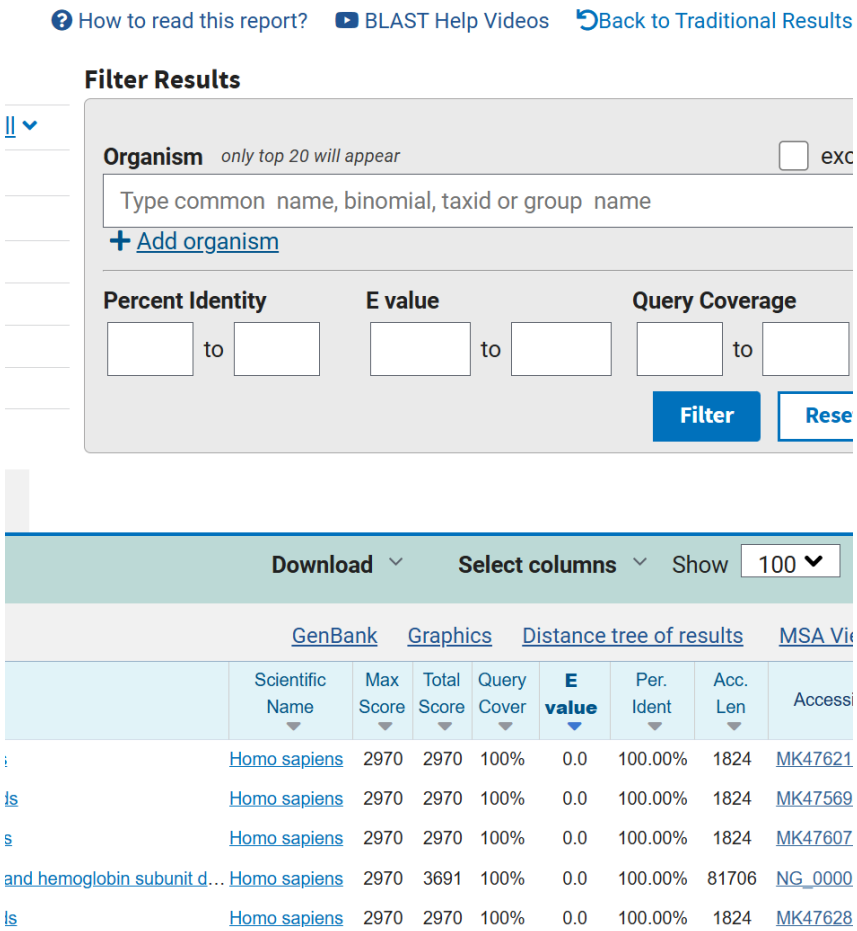
1. BLASTX
   - DNA query but search over proteins seq.
   - The DNA is translated with all 6 RF
   - To find potential protein products encoded by the query
2. TBLASTN
   - Protein seq. es query but search over DNA
   - The DNA is translated with all 6 RF
   - To find potential genes coding for the query protein

Other components of BLAST

3. TBLASTX
   - DNA query but search over DNA seq. but both are translated into protein seq. considering all 6 RF
   - Matched over proteins so 36 comparisons
   - To find nucleotide sequences similar to the query based on their coding potential

Blast on NCBI

- Significance
  - ***E-value*** - expected number of hits with the observed score or higher that would be seen by chance
  - The lower the better
  - Can be also interpreted as False positive rate
  - Depends on size of the database and size of the query sequence
  - Its interpretation in terms of true homology depends on the query
  - Default threshold: 0.05 , can be higher for shorter sequences and lower for longer sequences

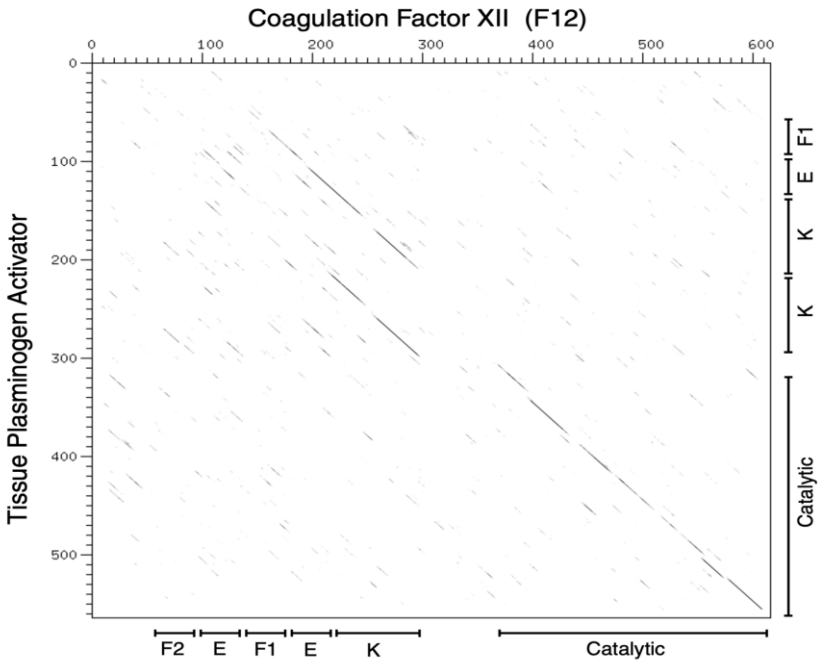


Blast on NCBI

- Demo:
  - Search NCBI for homologues of the TP53 protein in mouse
  - Search NCBI for a short part of TP53 protein in mouse
  - Compare hits and E-values among the 2 searches

FASTA - “Fast All”

- Idea:
  - Find “good diagonals” and extend local matches on these diagonals
- Steps:
  - Identify short matches between the two sequences (length ***ktup***)
  - Identify diagonals with a high density of short matches
  - Extend matches along these diagonals to expand the matches
  - Join ungapped regions using gaps
  - Realign the area using Smith-Waterman
  - Larger ***ktup*** - faster due to few initial matches, but may miss true alignments



FASTA on EBI

- https://www.ebi.ac.uk/jdispatcher/sss/fasta
- Similar to BLAST on NCBI but using FASTA algorithm
- Parameters:
  - Scoring, ***ktup***, significance thresholds

FASTA on EBI

- Demo:
  - Search EBI for homologues of the TP53 protein in mouse
  - Search EBI for a short part of TP53 protein in mouse
  - Compare hits and E-values among the 2 searches, and with BLAST results